

Q1. The dataset has 2930 houses and 82 columns. SalePrice goes from about 13,000 to 755,000 dollars, most houses are between 130,000 and 213,000, median is 160,000. It's not symmetric, a few very expensive houses pull the average up. A lot of missing values are not really missing, they mean the house doesn't have that thing, like no pool, no alley, no fence. Lot Frontage is the only column with real missing data. Overall Qual is split from 1 to 10 and that's useful later for the split. One thing we saw: some very big houses sold cheap, so there are a few outliers.

Q2. The features most linked to price are Overall Qual, Gr Liv Area, Exter Qual, Kitchen Qual, Garage Cars, Garage Area, Total Bsmt SF and 1st Flr SF. We saw this with correlation numbers and with boxplots: price goes up clearly with quality and changes a lot depending on the neighborhood too.

Q3. We split the data 80% train and 20% test, and we stratified on Overall Qual so both sets have the same proportion of each quality level. The test set has to stay untouched until the end because it's supposed to represent new houses the model never saw. If we used it earlier, the results would look better than they really are.

Q4. For missing values, when NaN means the house just doesn't have that feature (no garage, no basement, no pool, etc), we filled it with "None". For Garage Yr Blt we used Year Built when there's no garage. For categorical columns that have a real order (like quality: Po, Fa, TA, Gd, Ex) we used ordinal encoding so the order is kept. The other categorical columns without order were one-hot encoded. Numeric columns got median imputation and scaling. All of this was learned only on train data then applied to test, so we don't leak test information into training.

Q5. We dropped Order and PID because they're just IDs. We created 4 new features: Total_SF (total surface), House_Age, Total_Bath, and Has_Pool. Total_SF and Total_Bath turned out useful (high correlation), Has_Pool not really. We also removed 14 columns that had almost no correlation with price.

Q6. Results on test data:

Linear Regression: RMSE 32,149, R^2 0.845

Decision Tree: RMSE 34,206, R^2 0.825

Random Forest: RMSE 24,568, R^2 0.910

Random Forest is the best one. Decision Tree looks good on train (almost 0 error) but bad on test, so it's overfitting.

Q7. We could try a log transformation on SalePrice since it's skewed, check the outliers we found in Q1, try combining features like quality and size together, and maybe limit the depth of the Decision Tree since it overfits a lot.